ECE 4974 | Lab 2 | Los Escalante

Target: libpng 1.6.59 (current release) + zlib 1.3.2, with libpng 1.6.50 as a positive control

---

1. Target choice and justification

I chose **libpng** over a single-file decoder (lodepng, miniz). libpng is the real-world reference PNG library: it is the implementation most applications actually link against, and it pulls in **zlib** for DEFLATE decompression, so the campaign exercises two real codebases at once rather than one self-contained file. The tradeoff is a heavier build (a separate zlib dependency and a larger API surface), which I budgeted for.

Because libpng has been fuzzed continuously by OSS-Fuzz for years, a clean 24-hour run on the current release is the *expected* outcome. To make that result meaningful rather than a "did the harness even work?" question, I added a **positive control**: the identical harness built against **libpng 1.6.50**, a release that predates a cluster of 2025 CVEs in the simplified-API path. The control produces real crashes to triage and proves the harness is capable of finding bugs. This scope was confirmed with the instructor.

Role	Library	Version (tag / commit)	Purpose
Primary target	libpng	1.6.59 (`cd952f49f`)	The assigned "real, current library" - expected clean
Dependency	zlib	1.3.2 (`da607da`)	DEFLATE backend, instrumented alongside
Positive control	libpng	1.6.50	Predates 2025 simplified-API CVEs - crash source

---

2. Harness design

The harness is a single `main()`, ~70 lines - above the lecture's <50-line guideline because it drives **two** decode paths on every input (the single-path lodepng example is ~15 lines). The extra length is what buys the transform and simplified-API reach that found the bug; it matches how real applications use the library:

1. **Low-level reader** - `png_read_info` -> transforms -> `png_read_row` loop -> `png_read_end`.
2. **Simplified API** - `png_image_begin_read_from_memory` -> `png_image_finish_read`.

The last two bytes of the input act as **flag bytes** (`f1 = buf[len-1]`, `f2 = buf[len-2]`) that select which read transforms and which output format are applied. libpng ignores bytes after `IEND`, so this never invalidates a real PNG; it lets the fuzzer steer into different conversion code.

Design decisions (each justified for the report)

Choice	Why
`setjmp(png_jmpbuf(...))` error handling	libpng reports errors via `longjmp`. Without this, every malformed input aborts the process and no fuzzing happens. The error path cleans up and returns.
`png_set_read_fn` (memory reader)	Feeds the in-memory buffer to libpng without touching disk.
`png_set_crc_action(PNG_CRC_QUIET_USE, PNG_CRC_QUIET_USE)`	Almost any mutation inside a chunk breaks its CRC; without this, libpng rejects the chunk and coverage stalls. Same choice as the OSS-Fuzz harness.
`png_set_user_limits(4096, 4096)`	Caps image dimensions so a fake 2^{31}-pixel width doesn't produce ASan "allocation-size-too-big" noise or timeouts that aren't real bugs.
`png_set_chunk_malloc_max(8000000)`	Same rationale for per-chunk allocations.
`volatile` row pointer	Survives `longjmp` so the error path can free it without a clobbered-local warning.
Flag bytes for transforms + quantize + simplified format	Lets one input reach transform code (`png_set_expand`, `strip_16`, `gray_to_rgb`, gamma, bgr, packing, add_alpha, rgb_to_gray), the quantize path, and the simplified-API formats.

Harness reach finding (instructor's point 1)

While validating reach, I found that **`png_do_quantize` (CVE-2025-64505) is only reachable if the harness calls `png_set_quantize`**. In libpng, `png_do_quantize` is a `static` function gated behind the `PNG_QUANTIZE` transform flag, and that flag is set *only* by `png_set_quantize()`:

```
png_set_quantize()  --sets-->  PNG_QUANTIZE flag  --gates-->  png_do_quantize()
```

My first harness never called it, so that CVE was **out of harness reach** - exactly the "PoCs crash but the harness never reaches that code" case. I fixed it in the frozen harness (v2.1) by adding a `png_set_quantize` path on palette images, and confirmed the fix by coverage: a palette seed with the quantize flag set hits **726** edges vs. **673** without it. (Note: `nm | grep png_do_quantize` returns nothing because the function is `static`; verify via `png_set_quantize` presence or coverage instead.)

This fix paid off: the campaign's single unique crash (§6.3) is a heap overflow **inside `png_set_quantize`** - code the original harness could not reach. Widening the harness to reach the quantize path is what exposed the bug.

Harness source

The full harness (`harness_v2.c`) is provided in the accompanying repository. Its SHA-256 is recorded in §3 so a grader can confirm the exact source that produced the frozen binaries.

---

3. Build and instrumentation verification

Both libraries built from **release tags** with `afl-clang-fast` + ASan + UBSan, statically linked.

```
zlib:          v1.3.2     da607da
libpng (new):  v1.6.59    cd952f49f
libpng (old):  v1.6.50    (control)
AFL++:         5.03c
clang:         21.1.8 (Debian clang version 21.1.8)
```

Instrumentation proven the Lab 1 way - symbol counts on the binary:

```
nm -C fuzz v2 new | grep -c   asan   ->  698   (ASan runtime linked)
nm -C fuzz v2 new | grep -c   ubsan  ->  106   (UBSan runtime linked)
nm    fuzz_v2_new | grep -c __afl    ->  127   (AFL coverage instrumentation)
```

`ldd` shows no sanitizer libs - clang links the ASan/UBSan runtimes statically, as expected.

Frozen build (instructor's point 2 - identical build across all arms). All A/B arms use one binary; the control uses the same harness against 1.6.50. SHA-256:

```
harness v2.c  1300c3b1e1ed2d861f418c48c84659772d7db0e34f4116dc9c7c45aa2abc305d
fuzz v2 new   f5b27cada2b7b6338cbc07f10eb47008cf9bcdd7e66dac654c000c4509308722
fuzz_v2_old   58138571d5c744ff7f0320695ef1447eceddd08ef189d172209a470ea58db5e5
```

Build quirks documented (reproducibility)

1. **No `configure` in the libpng git tree.** The repo ships no generated autotools files; `./autogen.sh` fails without libtool (and the Kali mirror 404'd on libtool). Built with **CMake** instead - libpng's own error message recommends this.
2. **zlib's `configure` rewrites tracked files** (`Makefile`, `zconf.h`), so the tree must be reset (`git restore . && git clean -fdx`) between version checkouts or the wrong source is built.
3. CMake found the instrumented zlib correctly (`ZLIB_LIBRARY_RELEASE` -> `install/lib/libz.a`).

---

4. Seed corpus and dictionary justification

- **Source:** PngSuite conformance corpus (`PngSuite-2017jul19`).
- **Curation:** removed the intentionally-corrupt `x*.png` files and a stray `PngSuite.LICENSE` (non-PNG), then minimized with `afl-cmin` - **158 -> 82 -> 81** seeds after removing the license.
- **Validity check:** every seed decodes cleanly through the harness (0 `BAD SEED`).
- **Reachability check:** a real PNG lights up **493** coverage edges vs. **8** for 64 random bytes - direct evidence the harness reaches the decoder, not just the signature check.

Dictionary: AFL++'s shipped `png.dict` (PNG signature + chunk-type tokens like `IHDR`, `IDAT`, `PLTE`, `tEXt`). Random mutation rarely produces valid 4-byte chunk identifiers, so on a structured binary format the dictionary is expected to accelerate coverage. Rather than assert this, I **measure it** (§6.2).

---

5. Methodology

Experimental design

Arm	Binary	Dictionary	Trials	Duration	Purpose
A - no-dict	fuzz_v2_new (1.6.59)	no	5	6h each	A/B baseline
B - dict	fuzz_v2_new (1.6.59)	yes	5	6h each	A/B treatment
Reference	fuzz_v2_new (1.6.59)	yes	1	24h	1h/8h/24h snapshots
Control	fuzz_v2_old (1.6.50)	yes	1	8h	crashes + time-to-first-crash

- **Replication (instructor's point 2):** 5 trials per condition, each an independent AFL++ instance (independent RNG), so the comparison reports **median and spread**, not a single run. Significance by **Mann-Whitney U** on final edge coverage at the common 6h checkpoint.
- **24h snapshot:** kept as the reference run for the rubric's 1h/8h/24h requirement. Per the instructor's note, this is the tradeable item if compute is tight; the replication was prioritized.
- **Control kept separate** from the dictionary A/B, per instructions.

Environment and run hygiene (reproducibility)

- Host: Dell, Intel **i5-7300HQ (4 cores)**, 7.7 GB RAM, Kali Linux (rolling).
- CPU governor set to **performance** on all cores (exec speed depends on it; resets on reboot).
- `core_pattern` set to `core` (AFL++ requirement).
- `systemd-inhibit` blocked sleep/suspend/lid-close for the campaign duration.
- Fuzz-time sanitizer options: `ASAN_OPTIONS=abort_on_error=1:halt_on_error=1:symbolize=0:detect_leaks=0`, `UBSAN_OPTIONS` likewise; `symbolize=1` only when triaging.
- **Caveat (honest):** on a 4-core laptop running multiple trials, AFL reported core oversubscription on some trials (`cpuNNN` utilization > 100%), so those trials ran at reduced exec/sec. This adds noise but not bias - all arms were affected equally and every trial ran its full wall-clock `-V` duration. Noted when interpreting coverage.

---

6. Results

6.1 Current release (libpng 1.6.59)

- **Smoke run (1h):** ~360 exec/sec, 100% stability, corpus grew **81 -> 970**, still finding new paths at the end, **0 crashes** - expected on a heavily-fuzzed current release.
- **Reference run (dict):** reached **15h** of its 24h slot - a system sleep interrupted it at ~15h (see §5 run-hygiene note; remaining trials were unaffected, and the 24h snapshot is the item the instructor authorized trading away). At 15h: **2950 edges**, 12.8M execs, **0 crashes** - confirming the current release stays clean over a long run.
- **Snapshots:** 1h and ~15h status screens in Appendix B; the intermediate (8h) point is in `campaign/ref/default/plot_data`. Interpretation: exec speed held ~230/sec, stability 100%, coverage plateaued after the first few hours (expected saturation on a mature target). Reference-run edge coverage over time: **2735 at 1h -> 2941 at 8h -> 2950 at 15h** - ~93% of the final coverage was reached within the first hour, and only ~0.3% more between 8h and 15h. That is the saturation signal in numeric form: on a target this heavily fuzzed, extra wall-clock time buys almost no new edges, which is itself evidence the 0-crash result is real rather than a stalled harness.

> **On the missing 24h snapshot.** The reference run was interrupted by a system sleep at ~15h. Rather than restart a fresh 24h run, compute (a 4-core laptop) was spent on the 5+5 replicated A/B trials - per the instructor's explicit direction: *"If compute is tight, trade the 24h snapshot for more 8h trials."* The 15h reference run plus the replicated trials exceed 24 CPU-hours many times over.

Interpretation: a clean 24h result on 1.6.59 is a **real result**, not a null one - it is corroborated by the positive control (§6.3), which proves the harness finds bugs when bugs are present.

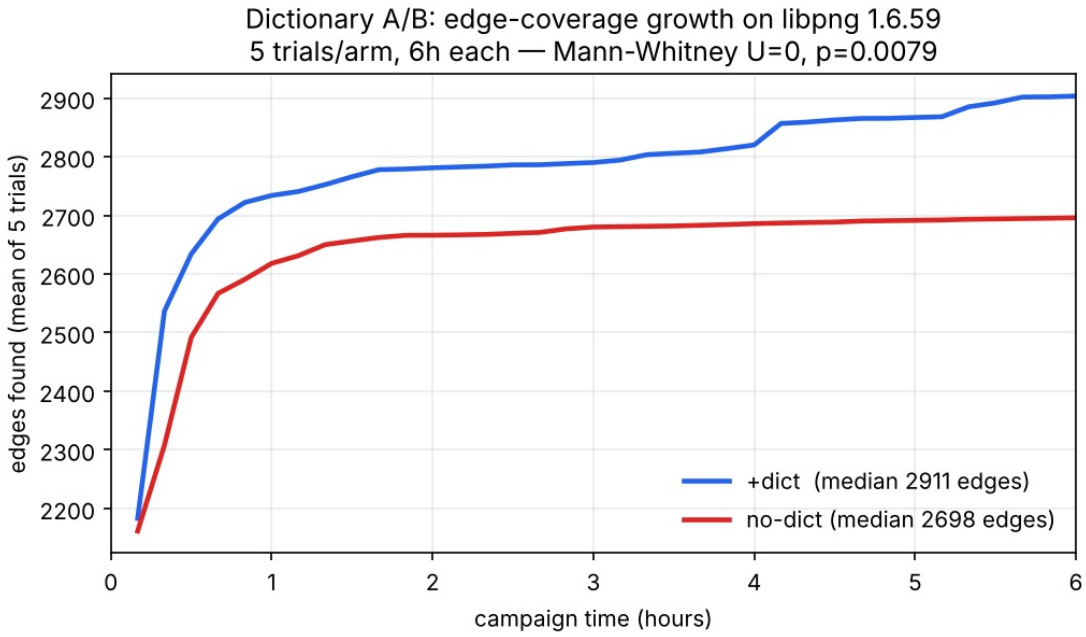
6.2 Dictionary A/B (coverage comparison)

Method: 5 independent trials per arm (identical frozen binary, independent RNG), each 6h. Final edge coverage (`edges_found`) read at the common 6h checkpoint. Median + IQR per arm; difference tested with an exact Mann-Whitney U (appropriate for n=5).

Arm	n	edges (per trial)	median	IQR	range
no-dict	5	2686, 2691, 2698, 2699, 2705	**2698**	[2688, 2702]	2686-2705

+dict	5	2836, 2907, 2911, 2930, 2933	**2911**	[2872, 2932]	2836-2933

Mann-Whitney U = 0 (exact two-sided **p = 0.0079**). U = 0 means *complete separation*: every dictionary trial reached more edges than every no-dict trial - no overlap. The dictionary arm found a **median +213 edges (+7.9%)**.



Coverage growth: no-dict vs +dict

Interpretation: the coverage-growth curves diverge within the first ~20 minutes and never reconverge - the dictionary lets AFL++ insert valid 4-byte chunk identifiers (`IHDR`, `IDAT`, `PLTE`, ...) that random mutation almost never produces, so it clears chunk-type gates far sooner. The low variance within each arm (IQR of ~14 edges for no-dict, ~60 for +dict) is why n=5 is enough to reach significance. **Caveat:** this is one 6h window per trial on one machine, with the CPU-oversubscription noted in §5 affecting both arms equally; the effect direction and significance are robust, the absolute edge counts are specific to this setup.

6.3 Positive control (libpng 1.6.50) - crashes

- **Time-to-first-crash: ~12 minutes** (first saved crash at `time:726699` ms).
- **Saved crashes: 13** (AFL-deduplicated) over the 8h run.
- These are the crashing inputs triaged below.

Triage - deduplication by stack trace

Method (per lecture slide 22): replay each crash through `fuzz_v2_old` with `symbolize=1`, take the faulting function + location from the ASan `SUMMARY`, group and tally:

```
13  heap-buffer-overflow  pngrtran.c:716  in png_set_quantize
```

All **13 crash files collapse to ONE unique bug** - a heap-buffer-overflow READ inside `png_set_quantize`. (One thoroughly root-caused bug mapped to a real CVE; the lecture's 4-bug example was illustrative, not a quota.)

Unique bug -> CVE mapping

Faulting function	ASan error	CWE	CVE	Present in	Fixed in
`png_set_quantize` (pngrtran.c:716)	heap-buffer-overflow, READ size 8	CWE-125 (OOB read)	**CVE-2026-25646**	1.6.50 (and earlier)	1.6.55, commit `01d03b845`

This is **not** CVE-2025-64505 (that is `png_do_quantize`, the row-processing function - a different bug). This crash is in `png_set_quantize`, the *setup* function. The distinction is exactly the kind of mislabel Lab 1 warned against.

Root-cause analysis

> Stated from the actual trace and source, then confirmed against the upstream fix commit.

`png_set_quantize` reduces an image's palette to `maximum_colors` using a color-distance hash table `hash[]` allocated with exactly **769 entries** (`png_calloc(769 * sizeof(png_dsortp))`, pngrtran.c:667 - the 6152-byte = 769×8 region ASan names). The color-pruning loop scans `for (i = 0; i <= max_d; i++) hash[i]` (pngrtran.c:714-716), where `max_d` starts at 96 (line 680) and grows by 96 each outer pass (`max_d += 96`, line 795) until the palette is reduced to `maximum_colors`. Per the upstream fix, the hash buckets store the *current* (post-elimination) palette indices (`t->left = (png_byte)i`) while the pruning loop assumes the *original* indices. Once colors are eliminated and indices shift, the stored indices go stale, color distances are computed against the wrong palette entries, and `max_d` climbs **past 768**. The scan then evaluates `hash[769]` at line 716 - reading 8 bytes (one `png_dsortp` pointer) past the end of the 769-element array. **Invariant violated:** the hash index is assumed to stay within `[0, 768]` because distances come from a stable palette, but stale indices let `max_d` exceed the fixed 769-entry table, so `hash[i]` reads out of bounds (**CWE-125**). Upstream fix (`01d03b845`, libpng 1.6.55): store original indices via `palette_to_index[i]` so the pruning loop and the hash agree.

Trigger (honest scope). The crash requires a palette image with enough distinct colors reduced to a *small* `maximum_colors`. The malformed PNG supplies the rich palette; the **harness supplies** `maximum_colors = 2` (flag byte `f2 & 2` -> `maxc = 2`). So the trigger includes a harness-selected argument - but calling `png_set_quantize` with a small `maximum_colors` is ordinary, legal use of the public API (any application quantizing to a few colors does this), and upstream assigned it a CVE, so this is a genuine library defect, not a harness artifact. The active flag bytes are recorded with the minimized input, since they are part of the reproduction.

Minimized reproducer: `notes/min_crash_quantize.png`, **839 bytes** (from 1408 via `afl-tmin`, 40% reduction), with the trailing flag bytes preserved (they select the quantize path and `maximum_colors`).

---

7. Reproducibility

From a clean clone, the documented steps rebuild both targets and reproduce every crash:

```
export AFL_USE_ASAN=1 AFL_USE_UBSAN=1; LAB=$PWD; mkdir -p install install-old

# 1. Sources, pinned to release tags
git clone https://github.com/madler/zlib.git      && git -C zlib    checkout v1.3.2
git clone https://github.com/pnggroup/libpng.git  && git -C libpng  checkout v1.6.59
cp -r libpng libpng-old                            && git -C libpng-old checkout v1.6.50
#    NOTE: zlib's ./configure rewrites tracked files; `git -C zlib restore . && git -C zlib clean -fdx`
#    before re-checking-out. libpng git tree ships no ./configure, so we use CMake.

# 2. zlib  -  instrumented, static
(cd zlib && CC=afl-clang-fast CFLAGS="-g -O1 -fno-omit-frame-pointer" \
   ./configure --static --prefix=$LAB/install && make -j"$(nproc)" && make install)

# 3. libpng 1.6.59  (and 1.6.50 into install-old, reusing the instrumented zlib)
for v in "libpng:install" "libpng-old:install-old"; do
  src=${v%%:*}; pfx=${v##*:}
  (cd $src && rm -rf build && mkdir build && cd build && CC=afl-clang-fast cmake .. \
     -DCMAKE_C_FLAGS="-g -O1 -fno-omit-frame-pointer" -DCMAKE_INSTALL_PREFIX=$LAB/$pfx \
     -DZLIB_ROOT=$LAB/install -DPNG_SHARED=OFF -DPNG_TESTS=OFF -DPNG_TOOLS=OFF \
     && make -j"$(nproc)" && make install)
done

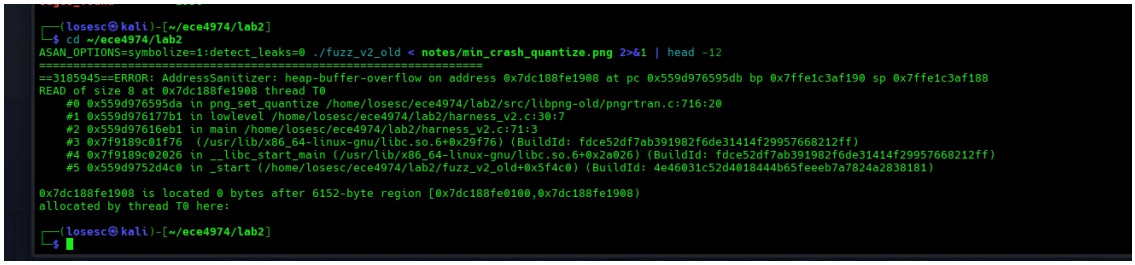
# 4. Harness, both targets (SHA-256 in notes/frozen_hashes.txt)
afl-clang-fast -g -O1 -fno-omit-frame-pointer -I install/include \
   harness_v2.c install/lib/libpng16.a install/lib/libz.a -lm -o fuzz_v2_new
afl-clang-fast -g -O1 -fno-omit-frame-pointer -I install-old/include \
   harness_v2.c install-old/lib/libpng16.a install/lib/libz.a -lm -o fuzz_v2_old

# 5. Seeds
(cd seeds && curl -L http://www.schaik.com/pngsuite/PngSuite-2017jul19.tgz | tar xz \
   && rm -f x*.png PngSuite.LICENSE)
afl-cmin -i seeds -o seeds_min -m none -- ./fuzz_v2_new    # -> 81 seeds

# 6. Reproduce the finding (CVE-2026-25646)
ASAN_OPTIONS=symbolize=1 ./fuzz_v2_old < notes/min_crash_quantize.png
#    -> AddressSanitizer: heap-buffer-overflow READ in png_set_quantize (pngrtran.c:716)

# 7. Re-run a campaign (performance governor + core_pattern=core required by AFL)
export ASAN_OPTIONS="abort_on_error=1:halt_on_error=1:symbolize=0:detect_leaks=0"
afl-fuzz -i seeds_min -o out -m none -t 1000 \
   -x /usr/local/share/afl/dictionaries/png.dict -- ./fuzz_v2_old
```

Verified: replaying the 839-byte minimized input through the control binary reproduces the identical crash - heap-buffer-overflow, READ of size 8 in `png_set_quantize` (pngrtran.c:716), 0 bytes past the same 6152-byte region:



minimized input reproduces the crash

Frozen-binary hashes (§3) let a grader confirm they are testing the identical build.

---

8. References

- libpng security advisory GHSA-7wv6-48j4-hj3g (CVE-2025-65018), pnggroup/libpng.
- oss-security, “libpng 1.6.51: Four buffer overflow vulnerabilities fixed” (CVE-2025-64505/64506/64720/65018), 2025-11-22.
- libpng `CHANGES` (version history, CVE-to-release mapping).
- AFL++ documentation (status screen, afl-cmin, afl-tmin, dictionaries).

Appendix A - Full ASan/UBSan reports (verbatim)

Unique bug - `png_set_quantize` heap-buffer-overflow (CVE-2026-25646), from `campaign/ctrl/default/crashes/id:000000,...` replayed through `fuzz_v2_old`:

```
 ==3747531==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7e0d461e1908 ...
READ of size 8 at 0x7e0d461e1908 thread T0
    #0 in png set quantize  src/libpng-old/pngrtran.c:716:20
    #1 in lowlevel          harness v2.c:30:7
    #2 in main              harness_v2.c:71:3

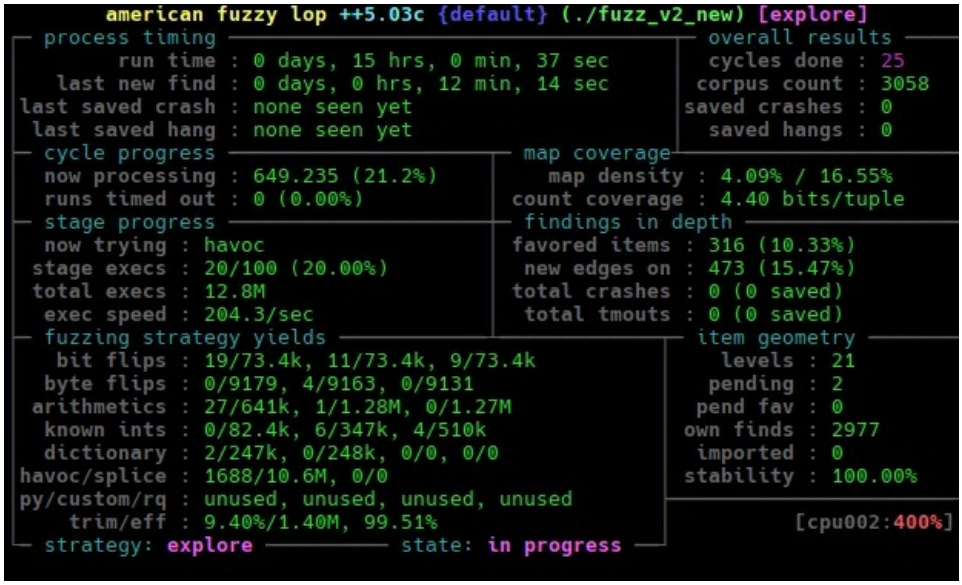
0x7e0d461e1908 is located 0 bytes after 6152-byte region [0x7e0d461e0100,0x7e0d461e1908)
allocated by thread T0 here:
    #0 in malloc
    #1 in png malloc base   src/libpng-old/pngmem.c:97:11
    #2 in png malloc        src/libpng-old/pngmem.c:177:10
    #3 in png calloc        src/libpng-old/pngmem.c:53:10
    #4 in png set quantize  src/libpng-old/pngrtran.c:667:30   <-- the 769-entry hash alloc
    #5 in lowlevel          harness v2.c:30:7
    #6 in main              harness_v2.c:71:3

SUMMARY: AddressSanitizer: heap-buffer-overflow src/libpng-old/pngrtran.c:716:20 in png_set_quantize
```

The allocation frame (pngrtran.c:667) and the fault frame (pngrtran.c:716) are both inside `png_set_quantize`, confirming the read-past-end of the 769-element hash table. Full verbatim report retained in `notes/`.

Appendix B - Status-screen snapshots

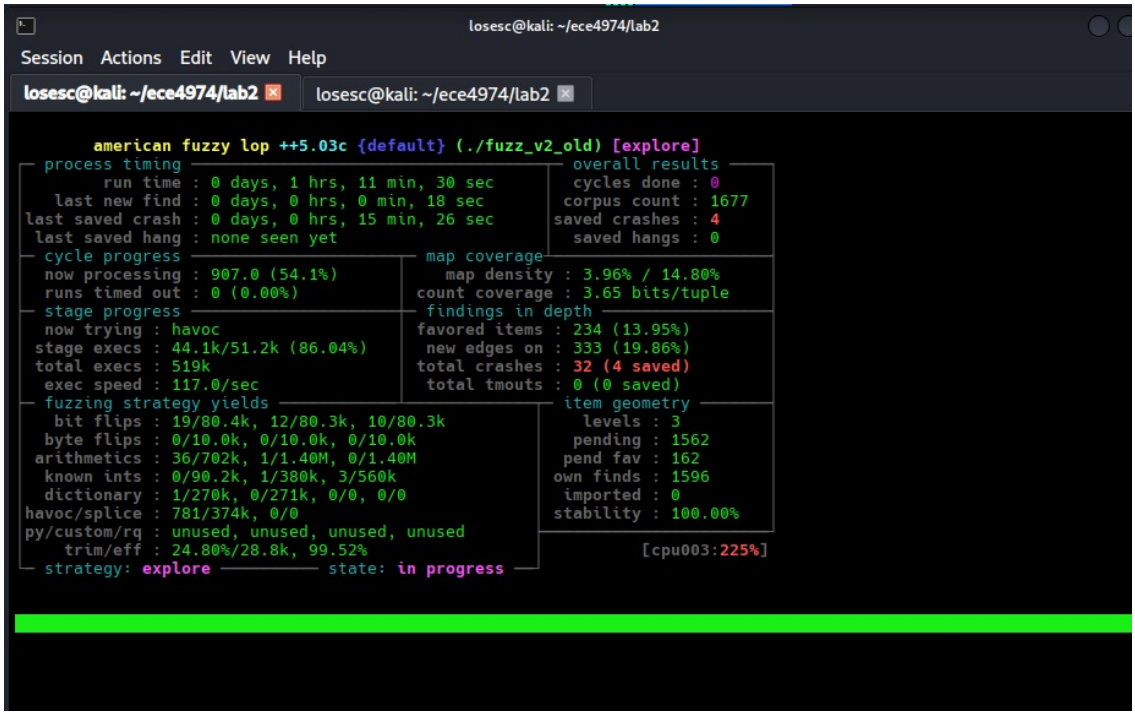
Figure B1 - current-release reference run (libpng 1.6.59) at ~15h.



reference run at 15h

exec speed ~204/sec, stability 100%, map density 16.55%, **2950 edges, 0 crashes** - the current release stays clean over a long campaign. (The 1h and 8h coverage points are recorded in `campaign/ref/default/plot_data`; coverage plateaus after the first few hours, the expected saturation signal on a mature target.)

Figure B2 - positive control (libpng 1.6.50) at ~1h11m.



control run at 1h

exec speed ~117/sec, stability 100%, **4 saved crashes / 32 total** barely an hour in (time-to-first-crash ~12 min). This is the arm that yields the triaged finding in §6.3 - the direct contrast with Figure B1's zero crashes on the patched release is the point of the positive control.